

SIBIDI

Sistema de Biblioteca Digital

Estructuras de Datos Lineales

Proyecto 5 - Ingeniería de Sistemas

Universidad - Ayacucho, Perú

¿Cómo está construido el Sistema?

</> Arquitectura Limpia (C++)

SIBIDI no utiliza bases de datos externas ni arreglos prefabricados (como `std::vector`).

Todo el manejo de la información (libros, usuarios y préstamos) se crea en tiempo real utilizando **memoria dinámica**, dándonos el control absoluto de cada *byte*.

Las Estructuras Clave

Un struct es como crear nuestro propio "molde" o cajita para guardar datos.

En el programa creamos tres cajitas fundamentales:

- ▶ `struct Libro`
- ▶ `struct Usuario`
- ▶ `struct Prestamo`

Las 3 Listas Enlazadas de SIBIDI



Lista Simple

Para guardar: Libros

Analogía: Una fila de personas. Cada persona usa su mano derecha para agarrar el hombro de quien está adelante.

Explicación: Cada libro (nodo) solo tiene un puntero (siguiente) que apunta al próximo libro.



Lista Circular

Para guardar: Usuarios

Analogía: Niños jugando a la ronda tomados de las manos. El último niño le da la mano al primero.

Explicación: El puntero siguiente del último usuario apunta de regreso al primero, cerrando un círculo infinito.



Lista Doble

Para guardar: Préstamos

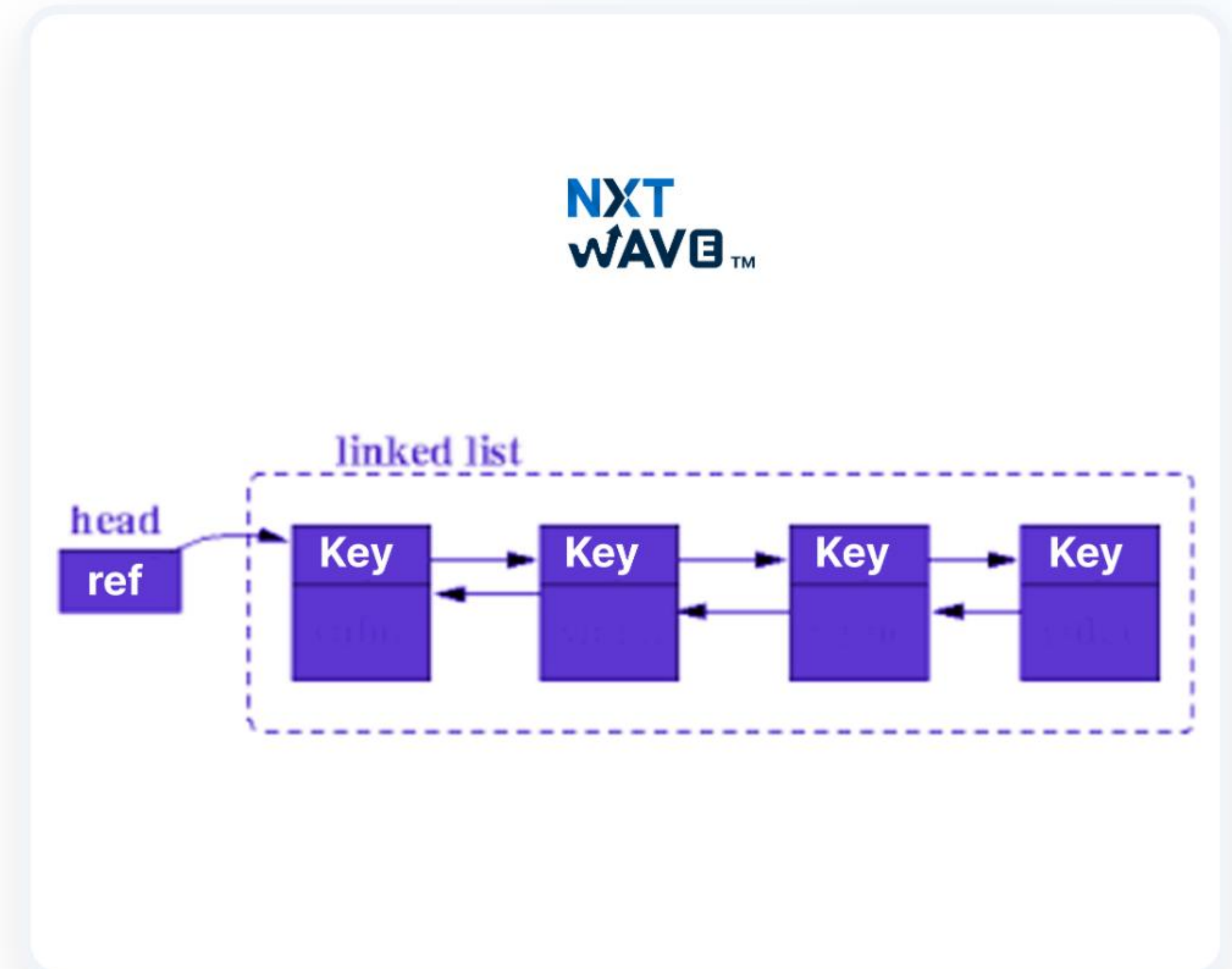
Analogía: Personas tomadas de ambas manos (izquierda y derecha).

Explicación: Tiene dos punteros (siguiente y anterior). Podemos recorrer el historial de préstamos hacia el futuro o hacia el pasado.

Punteros y Memoria Dinámica

Para que las listas funcionen, usamos la magia de los punteros:

- ▶ **La Cuerda Principal:** La clase `Biblioteca` tiene punteros (ej: `listaLibros`) que amarran al primer vagón de datos. Si se pierde ese puntero, se pierde toda la lista.
- ▶ **Crear datos:** Usamos `new Libro()` para pedir memoria prestada a la PC al instante en el que agregamos un libro nuevo.
- ▶ **Enganchar vagones:** Al nuevo libro le decimos `nuevo->siguiente = listaLibros;` uniendo la cadena.
- ▶ **Limpieza:** Todo lo creado con `new` debe borrarse usando `delete` para evitar que la RAM se llene (Memory Leaks).



El Concepto de Recursividad

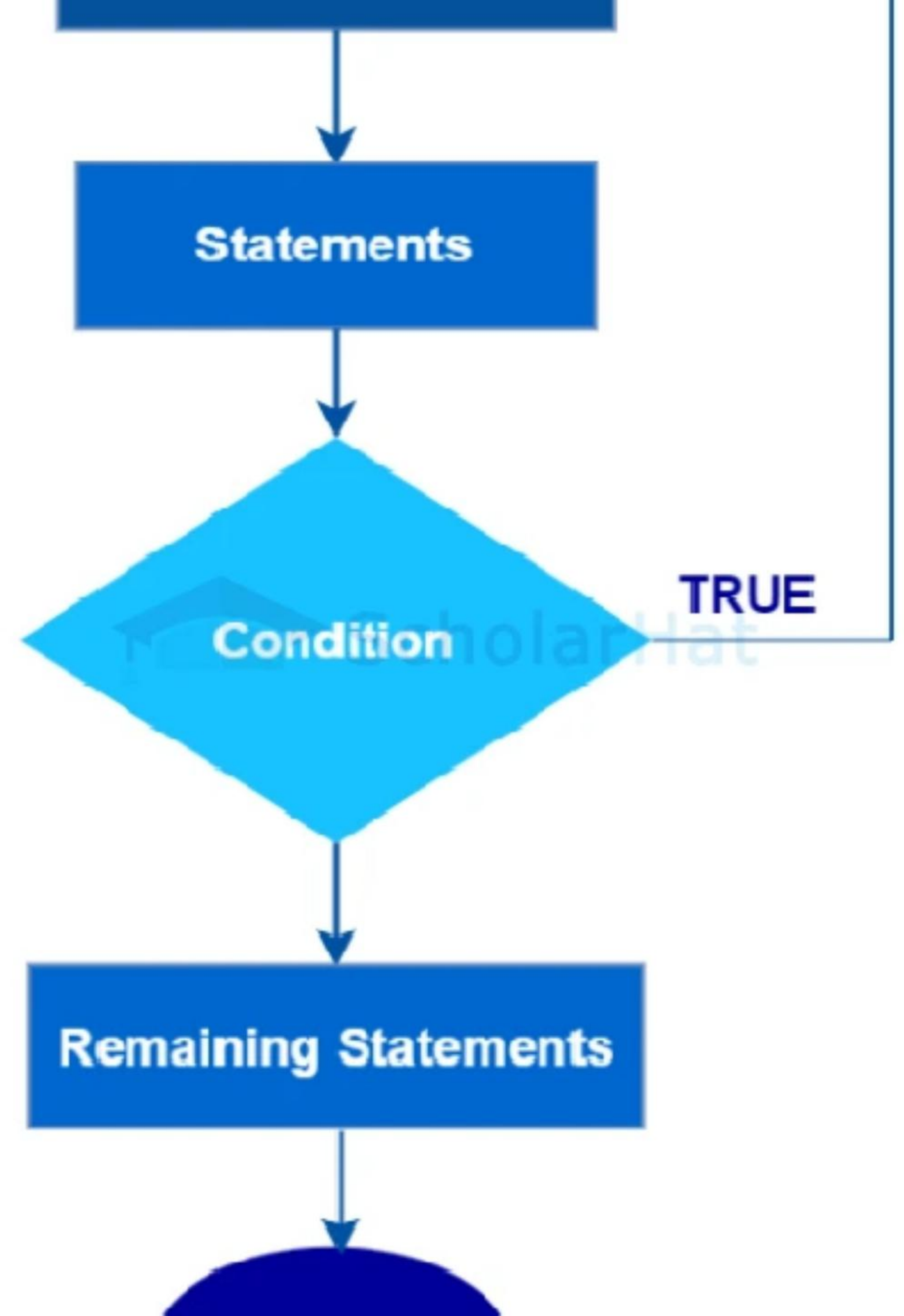
¿Qué es?

Es una función que *se llama a sí misma* para resolver un problema repetitivo, como si fuera un bucle, pero más elegante.

El "Caso Base": Toda recursividad debe saber cuándo parar para no ser infinita. En nuestro sistema es `if (nodo == nullptr) return;` (Si el vagón está vacío, detente).

Casos de uso en SIBIDI:

- `mostrarLibros()`: Imprime un libro y se llama a sí misma pasándole el *siguiente*.
- `contarDisponibles()`: Suma 1 o 0 y se llama a sí misma para sumar el resto.



Búsqueda Lineal

¿Cómo Funciona?

Cuando el usuario quiere prestar un libro, necesitamos verificar que el ISBN existe.

La Búsqueda Lineal comienza en el primer libro (vagón 1) y revisa uno por uno (`while temporal != nullptr`) hasta encontrarlo o llegar al final de la lista.

Ventajas y Desventajas

Ventaja: Es el algoritmo más fácil de entender y programar. No requiere que los datos estén ordenados previamente.

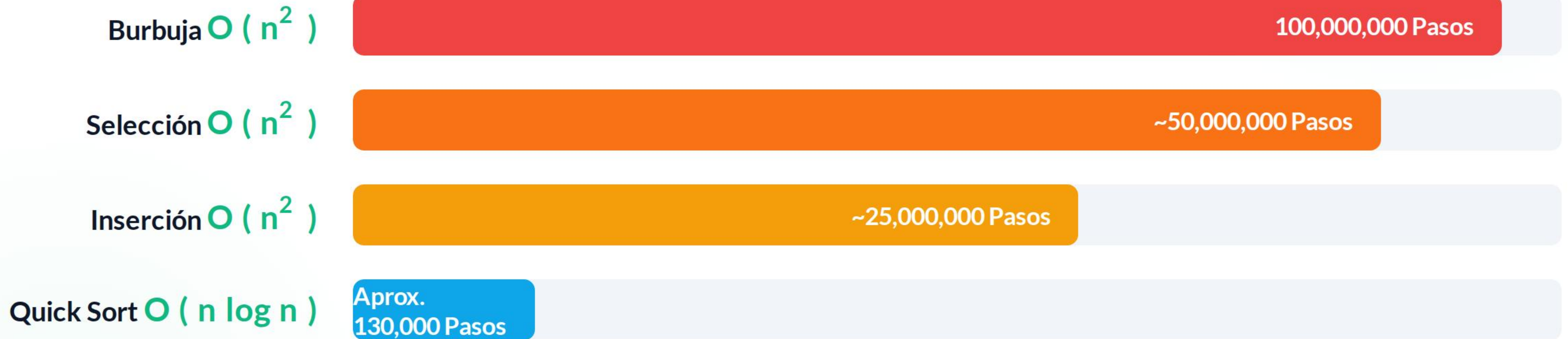
Desventaja: Su complejidad es $O(n)$. Si la biblioteca tiene 1,000 libros y el que buscamos está al final, ¡el programa dará mil pasos!

Algoritmos de Ordenamiento Implementados

Algoritmo	Analogía / ¿Cómo funciona?	Complejidad (Big o)
Burbuja (Bubble Sort)	Compara pares adyacentes; los libros "pesados" (ISBN mayor) caen al final de la lista como burbujas en el agua.	$O(n^2)$
Selección (Selection)	Busca el ISBN más pequeño de todos y lo pone en la posición 1. Repite buscando el segundo más pequeño.	$O(n^2)$
Inserción (Insertion)	Como jugar cartas: tomas una carta nueva y buscas el hueco exacto donde insertarla en tu mano ordenada.	$O(n^2)$
Merge & Quick Sort	Dividen la lista en pedazos pequeños (Divide y Vencerás), los ordenan y los unen. Los más rápidos del mundo.	$O(n \log n)$

El Porqué de Quick Sort (Ejemplo con 10,000 Libros)

Cantidad teórica de operaciones para ordenar todo el catálogo (Menos = Más Rápido)



Implementar **Quick Sort** y **Merge Sort** es crucial. Mientras que **Burbuja** colapsaría el programa con millones de pasos inútiles, **Quick Sort** lo ordena en milisegundos gracias a la técnica "Divide y Vencerás".

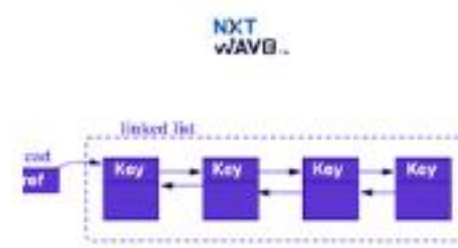
¿Preguntas?

Gracias por su atención.

</> Grupo 5 - Proyecto de Estructuras de Datos

Ayacucho, Perú

Image Sources



<https://d14qv6cm1t62pm.cloudfront.net/ccbp-website/Blogs/home/abstract-data-type-in-data-structure-image-9.png>

Source: www.ccbp.in



<https://dotnettrickscloud.blob.core.windows.net/img/data%20structures/3720230520122550.webp>

Source: www.scholarhat.com